

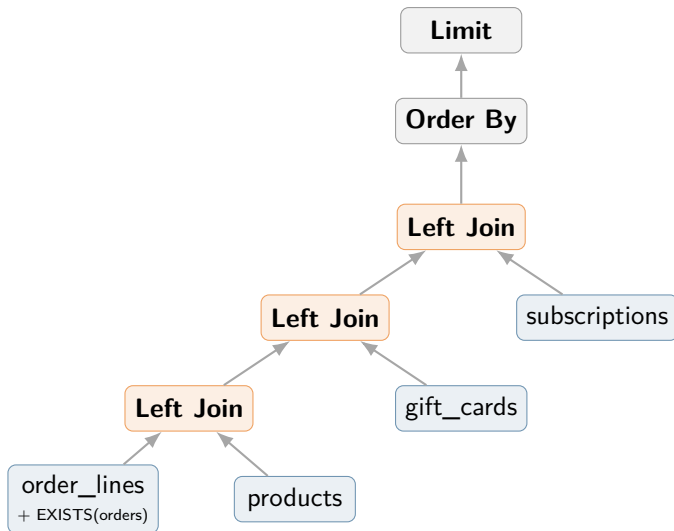
The Query

```
SELECT
    ol.id,
    COALESCE(p.name, g.name, s.name) AS item_name
FROM order_lines ol
LEFT JOIN products p
    ON ol.item_type = 'product' AND ol.item_id = p.id
LEFT JOIN gift_cards g
    ON ol.item_type = 'gift_card' AND ol.item_id = g.id
LEFT JOIN subscriptions s
    ON ol.item_type = 'subscription' AND ol.item_id = s.id
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.id = ol.order_id AND o.placed_at >= --2024\01\01)
ORDER BY ol.popularity
LIMIT 100
```

Typical optimiser choices

- NestLoop LEFT JOIN type because of PK index scan on the inner side

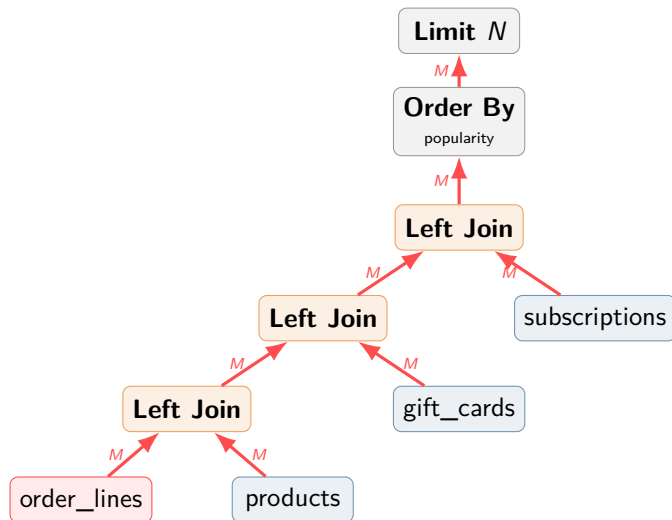
Query Plan Tree



Main Execution Cost Amplifiers

- Base table (`order_lines`) growth
- Ordering/grouping column of the base table not covered by an index
- Growth in the number of LEFT JOINS (target types)
- Unstable placement of the EXISTS filter

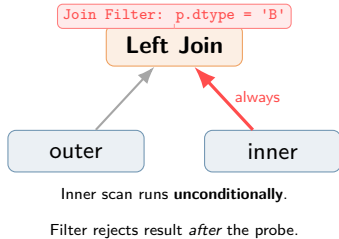
Worst case: base table growth + no index scan



Every edge carries M rows. Doubling M doubles number of probes on the inner side.

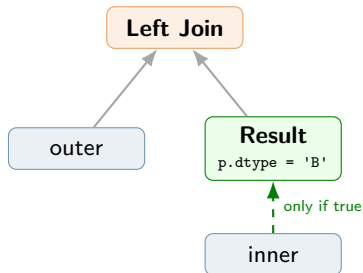
Concept: Result Filter (from pgsql-hackers)

Current: probe, then filter



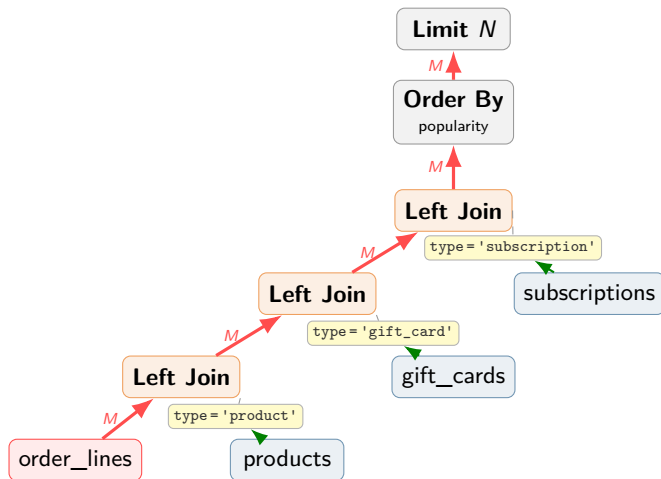
Thread on pgsql-hackers

Proposed: Result filter atop RHS



If false → returns $\emptyset$; inner scan *never launched*.

One-Sided Clause Guard



Guard checks discriminator *before* probing inner side. Each row probes only the matching table: $M_1 + M_2 + M_3 = M$.

Benefits of the Result Filter Approach

- **Fewer page accesses.** Each base row probes only the *matching* inner table instead of all N . Total index/heap page reads drop from $M \times N$ to M .
- **Safety net for missing indexes.** When no index covers the inner side, the executor would fall back to a sequential scan *per outer row*. The guard lets it skip the scan entirely for non-matching rows — turning an $O(M \times N \times |T_k|)$ catastrophe into $O(M \times |T_k|)$.